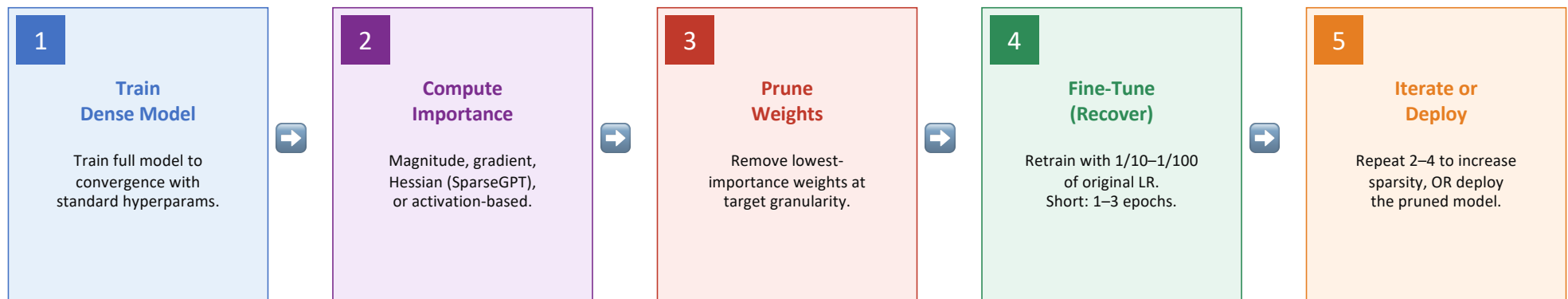


Iterative Pruning Cheat Sheet & Modern LLM Applications

Dr. Kaoutar El Maghraoui
Spring 2026

Iterative Pruning & Fine-Tuning: Cheat Sheet



↺ Iterate: increase sparsity each round
(e.g. 30% → 50% → 70% → 90%)

Key Parameters

Learning rate: 1/10 to 1/100 of original training LR

Fine-tune epochs: 1–3 per iteration (short-term); 5–20 final (long-term)

Sparsity schedule: Gradual: 30% → 50% → 70% → 90%

Pruning frequency: Every N training steps or once per epoch

Criterion: Magnitude (simple), Hessian (SparseGPT), Gradient (Movement)

Common Pitfalls

- Pruning too aggressively in one shot (always iterate!)
- Using full training LR for fine-tuning (causes divergence)
- Ignoring layer sensitivity (first/last layers are critical)
- Not measuring end-to-end speedup (sparse ≠ fast without HW support)
- Forgetting to combine with quantization (sparsity + quant = best)

Modern LLM Pruning Methods: Beyond Magnitude

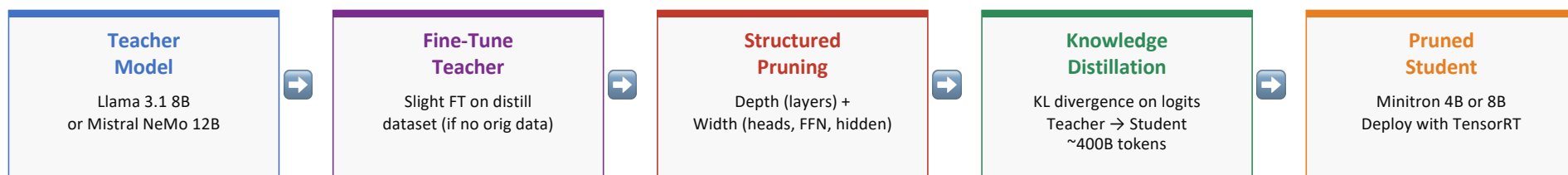
One-shot and training-aware methods that scale to billion-parameter models

SparseGPT <i>Frantar & Alistarh, 2023</i> One-shot unstructured pruning using approximate Hessian inverse. 50–60% sparsity with minimal perplexity increase. No retraining needed. Applied to: LLaMA-7B/13B/30B/65B, OPT-175B	Wanda <i>Sun et al., 2024</i> Pruning by Weights AND Activations. Multiplies weight magnitude $\times$ input activation norm. Simpler than SparseGPT, comparable results. Applied to: LLaMA-7B/13B/30B/65B, LLaMA-2	Minitron <i>NVIDIA, 2024</i> Structured depth+width pruning followed by knowledge distillation. 40x fewer training tokens than training from scratch. Applied to: LLaMA-3.1 8B $\rightarrow$ 4B, Mistral NeMo 12B $\rightarrow$ 8B	Sheared-LLaMA <i>Xia et al., ICLR 2024</i> Joint depth+width+head pruning with learned importance masks. Pruned models outperform same-size models trained from scratch. Applied to: LLaMA-2 7B $\rightarrow$ 1.3B/2.7B	MaskLLM <i>Fang et al., NeurIPS 2024</i> Learnable semi-structured (2:4) sparsity masks. Gumbel-softmax optimization of mask distribution. Superior to magnitude pruning. Applied to: LLaMA-2 7B, Nemotron-4 15B
--	--	---	--	---

Key trend: One-shot methods (SparseGPT, Wanda) enable pruning billion-parameter models in minutes on a single GPU. Structured methods (Minitron, Sheared-LLaMA) produce directly deployable smaller models.

NVIDIA Minitron: Prune + Distill Production Pipeline

From Llama 3.1 8B to Minitron 4B: 40x fewer training tokens than from scratch



Results: Minitron vs. Baselines

Model	Params	Method	Training Tokens	MMLU	vs. Baseline
Llama-3.1-Minitron-4B (width)	4B	Width prune + distill	~94B tokens	60.5	Beats Phi-2 (4B)
Llama-3.1-Minitron-4B (depth)	4B	Depth prune + distill	~94B tokens	58.7	Beats Gemma-2B
MN-Minitron-8B	8B	Width prune + distill	~380B tokens	69.5	Beats Llama-3.1-8B
Sheared-LLaMA-2.7B	2.7B	Joint prune + retrain	~50B tokens	49.3	Beats LLaMA-2-7B*

Takeaway: Pruning + distillation produces models that match or exceed same-size models trained from scratch, at 40x lower cost.

** On specific subsets. See original papers for full benchmarks.*

DeepSeek: Sparsity as a Core Architectural Principle

How DeepSeek uses Mixture-of-Experts (MoE) and attention sparsity to achieve frontier performance at lower cost

DeepSeek-V2 (2024) <i>Multi-Head Latent Attention+ MoE Architecture</i> MLA: Compresses KV cache by 90%+ — stores latent projections instead of full K,V MoE: 160 experts, top-6 active per token. Only ~21B active out of 236B total params Cost: Training cost reduced 40%+ vs. dense model at same quality	DeepSeek-V3 (2024) <i>Scaled MoE + Efficient Training</i> Scale: 671B total params, 37B active per token (256 experts, top-8) Training: 14.8T tokens, stable training with NO failures FP8: Mixed FP8 training for compute efficiency Cost: ~\$5.5M training cost — fraction of GPT-4 scale models	DeepSeek-R1 (2025) <i>Reasoning via RL + Distillation</i> RL: Reinforcement learning produces reasoning without supervised fine-tuning Distilled: R1-Distill models: 1.5B to 70B, pruned from Qwen/LLaMA bases Pruning: Adapt-Pruner: prune R1-Distill-1.5B → 1B, beats LLaMA-3.2-1B
---	--	--

Insight: DeepSeek's MoE = sparsity at the architecture level. Only 5–15% of params active per token. This is "pruning by design" — the router decides which experts to activate, like a learned pruning mask.

Modern Pruning Toolkit: From Research to Production

torchao

PyTorch

Semi-structured 2:4 sparsity
+ quantization. 1-liner API.
INT4+2:4 = 2.37x on LLaMA-3-8B.

```
pip install torchao
from torchao.sparsity import
SemiSparseLinear
```

SparseGPT

IST Austria

One-shot unstructured pruning.
Hessian-based weight update.
No retraining needed.

```
python run.py llama-7b
wikitext2 --sparsity 0.5
```

NVIDIA NeMo

NVIDIA

Full prune + distill pipeline.
Supports Llama, Mistral, Qwen.
Production-ready with TensorRT.

```
modelopt.torch.prune(
    model, mode='width',
    constraints=constraints)
```

Wanda

CMU

Weight × Activation pruning.
Simpler than SparseGPT.
Comparable quality.

```
python wanda.py
--model llama-7b
--sparsity_ratio 0.5
```

Recommendation for HPML Students

Start with **torchao** for 2:4 sparsity experiments (simplest API). Use **Wanda** for one-shot unstructured pruning on LLaMA. Scale to **NeMo** for full prune+distill pipelines in production.

Wanda — Prune by Weights × Activations

Sun, Liu et al. 2023 — a small weight times a huge activation is not small

Magnitude pruning asks: *which weights are biggest?* Wanda asks: **which weights carry the biggest signal through the layer?**

Wanda score: $S_{ij} = |w_{ij}| \cdot \|x_j\|_2$ (input activation norm over a small calibration set)

	w_1 0.1	w_2 0.5	w_3 0.2	w_4 0.8
weight $ w $	10	1	20	0.1
activation $\ x\ $	1.00	0.50	4.00	0.08
Wanda score				
Magnitude keeps	X	✓	X	✓
Wanda keeps	✓	X	✓	X

↑ tiny weight, huge activation → biggest contributor

How Wanda works in practice

Calibration only

~128 unlabeled sequences give $\|x\|$ statistics. No training data, no labels.

Per-row scoring

Weights are ranked inside each output row independently — matches dot-product structure.

One-shot, no backprop

No gradients, no fine-tuning. A drop-in upgrade over $|w|$ that beats it at every sparsity level.

SparseGPT — Prune and Compensate

Frantar & Alistarh 2023 — when you remove a weight, heal the survivors

One row of W : prune 2 of 5 weights, preserve Wx

original

0.50

0.30

0.10

0.40

0.20

magnitude
prune + leave

0.50

0.30

0

0.40

0

SparseGPT
prune + heal

0.58

0.33

0

0.45

0

+0.08

+0.03

+0.05

Output error: **Magnitude = loss** · **SparseGPT ≈ 0** (surviving weights absorbed the removed contributions)

The algorithm

Saliency (what to prune):

$$s_{ij} = w_{ij}^2 / [H^{-1}]_{jj}$$

Update (how to heal):

$$\Delta w_{ik} = - (w_{ij} / [H^{-1}]_{jj}) \cdot [H^{-1}]_{jk}$$

$$H = 2 \cdot XX^T + \lambda I$$

Hessian built from a small calibration batch X . Rows of W share H^{-1} $\rightarrow$ fast.

Per column, left to right:

1. Score each weight by $s = w^2 / [H^{-1}]$
2. Zero the weight with smallest score
3. Update surviving weights by Δw to cancel the error

Why it scales

Second-order

Uses Hessian curvature, not just $|w|$. Knows which weights are correlated and adjusts them.

One-shot

No fine-tuning. Prunes OPT-175B in ~ 4 hours on a single GPU.

Composable

Foundation for GPTQ. Supports unstructured, 2:4, and 4:8 sparsity patterns.